

# FinResearch AI

## Multi-Agent Financial Research System

### Complete Project Documentation

Written for first-time readers: what it is, how it works, and why every choice was made.

Companion to: PROJECT\_PRESENTATION.pdf (the evaluation deck)

October 2026

All figures in demo data are illustrative samples prepared for product demonstration.

# Contents

1. What is FinResearch AI? (problem, solution, audience)
2. Complete feature tour
3. Technology stack: what is used where, and why
4. System architecture and data flow
5. The seven agents (logic, code, files) + orchestrator
6. Data model (12 tables)
7. API reference (20 routes)
8. Website guide (every page)
9. Setup: local development and cloud deployment
10. Security, reliability and graceful degradation
11. Testing
12. Troubleshooting
13. Limitations and future roadmap
14. Glossary for beginners

# 1. What is FinResearch AI?

## 1.1 The problem it solves

Analysing a company the professional way means reading 100+ page annual reports, copying numbers into spreadsheets, checking for warning signs in the fine print, comparing several companies, and writing a summary. A single deep review can take hours. Most AI chat tools can summarize a document in seconds, but they invent numbers, cannot show where a figure came from, and forget everything between uploads.

## 1.2 The solution

FinResearch AI is a full-stack web application where a user uploads financial documents (PDF, DOCX or plain text) for one or more companies. The system automatically reads the documents, extracts 26 standard financial metrics, flags risks with exact source quotes, optionally builds a semantic search index, and then lets the user ask research questions, compare companies, and generate a polished analysis report - with every claim traceable back to a specific document line.

## 1.3 The three design promises

- Traceable: every metric and every risk stores the source text it came from. Answers carry [S1], [S2] style citations with document name, section and excerpt.
- Honest: deterministic code (regular expressions and maths) does the precision work; AI does the language work. If AI is unavailable, every feature degrades to a rule-based mode instead of failing.
- Runnable on a student budget: the entire stack runs on free tiers (Render web service, Neon Postgres, Groq AI) and the code keeps optional features (like embeddings) strictly optional.

## 1.4 Who it is for

- Students and analysts who want a first-pass view of a company fast.
- Investors comparing two or three companies on the same metrics.
- Anyone who wants AI help WITHOUT losing the evidence trail.

## 2. Complete feature tour

| Feature                    | What it does                                                                                                                                                         | Where to see it                     |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Accounts                   | Register and log in (bcrypt-hashed passwords, secure session cookies). Every user sees only their own data.                                                          | /register, /login                   |
| Document upload            | Drag-and-drop PDF/DOCX/TXT with company name, ticker, fiscal year and document type. Files are parsed and processed by a background pipeline.                        | Dashboard -> Documents -> Upload    |
| Agent pipeline             | Document Agent (index) -> Extraction Agent (metrics) -> Red Flag Agent (risks) -> Embedding Agent (optional vectors). Each step logs to an activity trail.           | Document detail page                |
| Metrics                    | 26 standard metrics per document (revenue, margins, EPS, ratios, cash flow...) each with the source line used as evidence.                                           | Metrics page, document detail       |
| Risk flags                 | Deterministic red-flag detection (auditor issues, going concern, accounting control, liquidity) plus metric anomalies and AI-added findings, all with quotes.        | Risks page                          |
| Cross-period trends        | When two years of the same company exist, the system compares them and raises trend risks (declining margins, rising debt, negative FCF...).                         | Risks page                          |
| Semantic search (optional) | Embedding Agent stores vector embeddings so research can use meaning-based retrieval. Without a provider it falls back to keyword search automatically.              | Document detail -> Build embeddings |
| Research Q&A               | Ask questions in plain English. The Research Agent decomposes compound questions, retrieves evidence, and answers with [S1] citations. It refuses to invent numbers. | Research page (chat)                |
| Benchmark                  | Compare up to several companies on revenue, profitability, leverage, liquidity and risk counts, with period-mismatch warnings.                                       | Benchmark page                      |
| Reports                    | Generate a structured analysis report (Key Financials table + narrative sections + disclaimer). Falls back to a deterministic report if AI fails. Download as text.  | Reports page                        |
| Sessions                   | Group documents into research sessions and scope questions to a session's documents.                                                                                 | Sessions page                       |
| Health                     | Public /api/health endpoint describing AI configuration and agent status - great for live debugging demos.                                                           | /api/health                         |

### 3. Technology stack: what is used where, and why

#### 3.1 Frontend

| Technology              | Where / for what                                   | Why this one                                                     | Why not the alternative                                                                   |
|-------------------------|----------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Next.js 16 (App Router) | The entire web app: pages, layouts, API routes     | One codebase, server rendering, fast deploys, file-based routing | Plain React (Vite) needs a separate backend server; PHP/Laravel lacks the React ecosystem |
| React 19                | All interactive UI (uploads, chat, tables, charts) | Component model, hooks, huge ecosystem                           | Vanilla JS becomes unmanageable at this UI complexity                                     |
| TypeScript              | Every source file                                  | Catches type errors before runtime; safer refactors              | Plain JS lets small mistakes reach production                                             |
| Tailwind CSS 4          | All styling                                        | Utility classes = fast, consistent, tiny CSS                     | Bootstrap/MUI: heavier bundle, fighting the framework to customise                        |
| Recharts                | Dashboard and benchmark charts                     | Declarative React charts                                         | Chart.js: imperative canvas, awkward with React state                                     |
| Framer Motion           | Page/panel animations                              | Declarative motion in React                                      | CSS transitions get clumsy for sequenced UI motion                                        |
| react-dropzone          | Upload panel                                       | Accessible drag-and-drop with validation hooks                   | Hand-rolled DnD is error-prone (browser quirks)                                           |
| react-hot-toast         | Notifications                                      | Lightweight toasts for agent status                              | Full alert libraries are overkill                                                         |

#### 3.2 Backend and data

| Technology                 | Where / for what                                   | Why this one                                                                                                     | Why not the alternative                                                                                                            |
|----------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Next.js API Routes (Node)  | 20 REST endpoints under /api                       | Same deploy unit as the UI; no CORS issues                                                                       | Separate Express/Django service = 2x deploy, auth sharing pain                                                                     |
| PostgreSQL (Neon)          | Primary database (12 tables)                       | Relational fit for financial data; SQL aggregation for benchmarks; ACID; free serverless tier                    | MongoDB: weak joins/aggregation for company-period analytics; no real need for document flexibility                                |
| Drizzle ORM 0.45           | All queries + migrations                           | TypeScript-first, SQL-like, light; schema is code                                                                | Prisma: heavier runtime and codegen; raw SQL: injection risk and no type checks                                                    |
| JSONB vectors              | document_chunks.embedding                          | Postgres JSONB stores vectors with zero extensions; cosine similarity computed in the app - fine to medium scale | pgvector: better at large scale but needs extension support and index tuning (a roadmap item)                                      |
| NextAuth (credentials)     | Register/login/sessions                            | Battle-tested, HTTP-only cookies, no third-party lock-in                                                         | Clerk/Auth0: paid at scale; rolling your own JWT auth is a security risk                                                           |
| bcryptjs                   | Password hashing                                   | Pure JS (no native build issues on Render); salted slow hashes                                                   | Plain SHA-256 is too fast to brute-force safely; native bcrypt breaks on some hosts                                                |
| Groq (openai/gpt-oss-120b) | All AI text work                                   | Generous free tier, very fast LPU inference, OpenAI-compatible API                                               | OpenAI: paid; Gemini: slower and its key errors are confusing (this project once mis-labelled them); local LLMs: need a GPU server |
| unpdf + pdf-parse          | PDF text extraction (two parsers, one as fallback) | unpdf is bundler-safe; pdf-parse proven; both are lightweight                                                    | Puppeteer/Playwright: headless Chromium needs hundreds of MB of RAM that free tiers lack                                           |

| Technology | Where / for what     | Why this one                | Why not the alternative                            |
|------------|----------------------|-----------------------------|----------------------------------------------------|
| mammoth    | DOCX text extraction | Clean, pure-JS .docx reader | LibreOffice conversion needs heavy system packages |

### 3.3 AI architecture choices (the interesting part)

| Decision                      | Why it is done this way                                                                                                                                                  | The alternative we rejected                                                                |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Hybrid deterministic + AI     | Numbers must be exact, so metrics and red flags use regex/math first; AI is used for language (narrative, synthesis) and as an ADDITIONAL reviewer, never the only truth | Pure-AI extraction hallucinates numbers; pure-rules cannot write narrative                 |
| Custom agents, not LangChain  | Each agent is a small, readable TypeScript file with explicit inputs/outputs and its own fallback - easy to explain, test and debug                                      | LangChain adds abstraction layers and dependencies that hide the flow and bloat the bundle |
| Provider-agnostic embeddings  | EMBEDDING_BASE_URL/ EMBEDDING_MODEL/ EMBEDDING_API_KEY point at any OpenAI-compatible endpoint (Gemini, Jina, Ollama...)                                                 | Hard-coding one vendor would lock the project and fail when that vendor is down            |
| Grounded answers only         | The Research Agent validates that numbers in its answer appear in retrieved source text; on any failure it returns verbatim evidence passages instead of guessing        | Trusting the LLM output blindly is how AI products lose credibility                        |
| Deterministic report skeleton | The Key Financials table in reports is rendered from stored metrics by code, not by the LLM                                                                              | Letting the LLM re-type numbers invites transcription errors                               |

### 3.4 DevOps

| Technology                | Where / for what                                                         | Why this one                                                 | Why not the alternative                                                                             |
|---------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Render (free web service) | Hosts the Next.js app (build: npm ci && npm run build, start: npm start) | Free tier, GitHub auto-deploy, env var UI, easy for students | Vercel: hobby limits and function quirks for this workload; a VPS costs money and needs maintenance |
| Neon Postgres             | Serverless Postgres with pooled + direct URLs                            | Free tier, autosuspend, connection pooling built in          | Local/RDS Postgres: always-on cost or not reachable from the cloud                                  |
| Render Cron Job           | Pings /api/worker/documents every 15 minutes with a secret               | Drains the background job queue reliably on a schedule       | Queue services (SQS, BullMQ/Redis): extra accounts, extra cost, extra failure points                |
| GitHub (git)              | Source control, PR reviews, deployment trigger                           | Industry standard; PR workflow keeps main stable             | Emailing zip files / shared drives is not version control                                           |
| Node 22 (.nvmrc)          | Pinned runtime version                                                   | Reproducible builds local and on Render                      | Floating default Node versions cause 'works on my machine' bugs                                     |

## 4. System architecture and data flow

### 4.1 The big picture

A single Next.js application serves both the UI and the API. The browser never talks to the database or the AI directly - all secrets stay on the server. Documents are processed by an asynchronous pipeline (a job queue in Postgres) so uploads stay fast and never time out.

- Browser (React) -- HTTP --> Next.js pages and API routes
- API routes -- Drizzle ORM --> PostgreSQL (Neon): users, companies, documents, chunks, metrics, risks, reports, chats, benchmarks, agent\_logs, jobs
- Agents -- HTTPS --> Groq (chat/completions, JSON mode) and an optional embeddings provider
- Upload creates a 'processing job'; the /api/worker/documents endpoint (cron every 15 min, or immediate kick) claims jobs atomically and runs the pipeline

### 4.2 Upload-time pipeline (runs once per document)

- 1. POST /api/documents validates auth, file type (pdf/docx/txt) and size, extracts text (two PDF parsers with fallback), creates the document row and enqueues a job.
- 2. Pipeline Orchestrator runs stages in order with failure isolation: Document -> Extraction -> Red Flag -> Embedding (optional).
- 3. A 'partial' result is allowed: if one stage fails, later stages still run and the document status explains what failed.
- 4. Every stage writes to agent\_logs (agent name, action, status, duration, details) - the visible activity trail on the document page.

### 4.3 On-demand agents (run when the user asks)

- Research Agent: question -> decomposition -> retrieval -> grounded answer with citations (POST /api/chat).
- Benchmark Agent: company set -> metric profiles -> comparisons + narrative (GET/POST /api/benchmark).
- Report Agent: company profile + metrics + risks -> structured markdown report (POST /api/reports).

### 4.4 Resilience design

- Job queue: atomic claim (UPDATE ... WHERE status='queued'), 15-minute stale-lock expiry, retry with backoff up to max attempts, batch size 5 per worker run.
- AI failures: every agent has a deterministic fallback (regex extraction, keyword search, local report, local benchmark).
- Embeddings are never required: no provider -> 'unavailable' status and keyword retrieval keeps research fully working.

## 5. The seven agents (logic, code, files)

*Agent files live in src/lib/agents/. The orchestrator (orchestrator.ts) runs upload-time agents; research/report/benchmark agents run on demand. llmClient.ts is the single door to the Groq API; aiStatus.ts tracks configuration and converts raw AI errors into user-safe messages.*

### 5.1 Document Agent - documentAgent.ts

Job: turn raw text into searchable chunks. It splits text into overlapping chunks of 1500 characters (200 char overlap), detects section headings (BUSINESS OVERVIEW, RISK FACTORS, MANAGEMENT DISCUSSION, FINANCIAL STATEMENTS, LIQUIDITY, RESULTS OF OPERATIONS, NOTES) with regular expressions, stores each chunk in document\_chunks with its section and position, and exposes hybrid search (keyword relevance + optional cosine similarity over embeddings).

Why 1500/200: a chunk must be small enough to quote precisely in an answer, but large enough to keep context; the overlap ensures a sentence that straddles a boundary exists complete in at least one chunk.

```
export function chunkText(text: string, chunkSize = 1500, overlap = 200): string[] {
  const chunks: string[] = [];
  let start = 0;
  while (start < text.length) {
    const end = Math.min(start + chunkSize, text.length);
    chunks.push(text.slice(start, end));
    start += chunkSize - overlap;
  }
  return chunks;
}
```

Key functions: chunkText, extractSections, processDocument, searchDocumentChunks, searchDocumentCollection.

### 5.2 Extraction Agent - extractionAgent.ts + extractionUtils.ts

Job: extract 26 financial metrics. It uses a hybrid strategy: (1) deterministic local parsing with a labelled-grammar (LABELS) regex table, capturing the matched source line as evidence; (2) an LLM pass over the first 15,000 + last 15,000 characters (documents are long, budgets are small); (3) mergeWithLocalMetrics - local values win wherever the label line is unambiguous, because regex on an exact label is more reliable than a model re-typing a number. If the AI is down, step (1) alone still fills the table.

```
// extractionUtils.ts -- the labelled grammar (excerpt)
revenue: /^s*(?:total net sales|total sales|total net revenues?|
  total revenues?|net sales|net revenues?|revenues?)s*[:\u2014-]/i

// extractionAgent.ts -- hybrid merge (excerpt)
extracted = mergeWithLocalMetrics(verified, extractMetricsLocally(content));
```

Numbers are normalised into a consistent shape: percentages as decimals (44.1% -> 0.441), money parsed from '\$383,285 million' style strings, fiscal year detected from 'Fiscal year: 2023' style lines. Every metric can store metric\_evidence: the exact line it came from - which is what powers the 'show source' experience and keeps report numbers verifiable.

### 5.3 Red Flag Agent - riskAgent.ts + analysisUtils.ts

Job: find risks in three deterministic layers plus one AI layer.

- Layer 1 - textual red flags (regex, with severity): qualified/adverse/disclaimer audit opinion or 'except for the effects of' (critical); any going-concern mention (critical); material weakness in internal control / restatement / non-reliance / accounting irregularities (high); debt covenant breach / default / liquidity shortfall / unable to meet obligations (high).
- Layer 2 - metric anomalies: negative margins, collapsing growth, negative free cash flow and similar threshold checks on the extracted metrics.
- Layer 3 - cross-period trends: if another fiscal year of the same company exists, detectFinancialTrendRisks compares them (declining margins, falling revenue, rising debt, negative FCF) and cites BOTH periods as evidence.
- Layer 4 - AI review: the LLM sees excerpts and may add material risks the rules missed - it is asked NOT to duplicate deterministic findings, and its output is stored with source text like everything else.

```
// analysisUtils.ts -- one of four deterministic patterns (excerpt)
{
  pattern: /\b(?:substantial doubt about.{0,100}going[- ]concern|going[- ]concern)\b/i,
  riskType: "Going Concern",
  severity: "critical",
  title: "Going-concern disclosure present",
}
```

Every finding stores risk\_type, severity, title, description, source\_text (the matched excerpt, capped at 6000 chars) and a recommendation - the Risks page can therefore always show the exact filing language behind a flag.

### 5.4 Embedding Agent - embeddingAgent.ts

Job: optionally build a semantic search index. For each chunk (batched 32 at a time, each truncated to 8000 chars) it calls any OpenAI-compatible /embeddings endpoint configured by EMBEDDING\_BASE\_URL / EMBEDDING\_API\_KEY / EMBEDDING\_MODEL, and stores the vector in JSONB with the model name. Search later computes cosine similarity in application memory. If the model name changes, chunks are re-embedded automatically (embeddingModel versioning). If no provider is configured the agent records 'skipped/unavailable' and research uses keyword retrieval instead - nothing else breaks.

```
// llmClient.ts -- provider-agnostic call (excerpt)
const response = await fetch(`${baseUrl}/embeddings`, {
  method: "POST",
  headers: { "content-type": "application/json", authorization: `Bearer ${apiKey}` },
  body: JSON.stringify({ model, input: inputs }),
});
```

This is why the same project runs with Ollama on a laptop (<http://localhost:11434/v1>) and with a cloud provider on Render - the agent does not care who serves the endpoint.

### 5.5 Research Agent - researchAgent.ts + analysisUtils.ts

Job: answer questions using ONLY the user's documents, with citations. The flow: (1) decomposeResearchQuestion splits compound questions into retrieval steps (e.g. 'compare margins' becomes one step per company); (2) each step searches the document collection - semantic search when embeddings exist, keyword search otherwise; (3) a relevance filter drops weak matches; (4) buildCitations labels evidence [S1], [S2]... with document name, section, chunk number and an excerpt; (5) the LLM synthesises an answer from the retrieved passages only; (6) validateGroundedResearchAnswer checks that numeric claims in the answer actually appear in the source excerpts; (7) if any AI step fails, the agent returns the verbatim passages clearly labelled as evidence instead of an answer.

```
// researchAgent.ts -- citation construction (excerpt)
citationId: `${index + 1}`

// the answer format the UI renders:
//   ...operating margin was 44.1% [S1]...
//   [S1] Apple FY2023 -- Results of Operations, chunk 2: "..."
```

The 'reasoning' field shows retrieval steps (which searches ran), not hidden chain-of-thought - honest and debuggable. This design is why the demo prompts 'quote the supporting source text' and 'compare X and Y and quote each figure' work so well.

## 5.6 Report Agent - reportAgent.ts + aiStatus.ts

Job: produce a structured analysis report: Key Financials table, Business Overview, Risk Assessment, Financial Analysis, Outlook and Disclaimer. The Key Financials table is rendered by code from stored metrics (renderKeyFinancialsSection) - the LLM is never trusted to re-type numbers. Narrative sections come from the LLM; if it fails, createLocalReport builds a complete deterministic report and the disclaimer states the reason via describeNarrativeFallbackReason, which maps raw errors (including stale GEMINI\_API\_KEY-era messages) to clean Groq guidance. Sections are spliced into the markdown with replaceOrInsertMarkdownSection so re-generation cannot corrupt the table.

```
// aiStatus.ts -- clean fallback disclaimer (excerpt)
export function describeNarrativeFallbackReason(error?: unknown): string {
  // maps raw AI errors to: name the real key (GROQ_API_KEY),
  // never leak 'Error:' prefixes or retired provider advice into reports
}
```

Reports are stored in analysis\_reports and can be downloaded from the Reports page as text/markdown.

## 5.7 Benchmark Agent - benchmarkAgent.ts

Job: compare companies on the same scale. It builds a metric profile per company (latest or selected fiscal period), normalises units, then compares revenue/growth, profitability (margins, ROE), leverage/liquidity (debt-to-equity, current ratio) and risk counts. The local engine can rank by ROE and handle missing values with N/A. The AI narrative is instructed to name fiscal periods beside figures, refuse to silently compare mismatched years, and never give buy/sell advice.

```
// benchmarkAgent.ts -- deterministic core (excerpt)
const ranked = [...context].sort((a, b) => Number(b.metrics?.roe || 0) - Number(a.metrics?.roe || 0));
```

## 5.8 Pipeline Orchestrator + job queue - orchestrator.ts

Job: run upload-time agents in order with failure isolation, and manage a durable job queue. Stage errors are collected, not fatal: a failed extraction does not block risk scanning, and embeddings never block anything. The queue uses an atomic claim (UPDATE document\_processing\_jobs SET status='processing' WHERE status='queued' AND next\_attempt\_at <= now()), a 15-minute stale lock so crashed runs can be retried, and a batch size of 5 per worker call.

```
// orchestrator.ts -- failure isolation (excerpt)
try { await extractFinancialMetrics(documentId, content, companyId); }
catch (error) { failures.push(`Extraction Agent: ${...}`); }
try { await scanForRisks(documentId, content, companyId); }
catch (error) { failures.push(`Red Flag Agent: ${...}`); }
// embeddings are optional: a failure only logs a warning
```

The worker endpoint `/api/worker/documents` is protected by `CRON_SECRET` and is triggered by a 15-minute Render cron job (and immediately after upload, so demo speed is instant).

## 6. Data model (12 tables)

| Table                    | Purpose                                   | Key columns (summary)                                                              |
|--------------------------|-------------------------------------------|------------------------------------------------------------------------------------|
| users                    | Accounts                                  | id, name, email, password_hash                                                     |
| research_sessions        | Grouping of documents for scoped research | id, user_id, title                                                                 |
| companies                | Company master data (shared lookup)       | id, name, ticker                                                                   |
| documents                | Uploaded files + processing state         | id, user_id, company_id, fiscal_year, content, processing_status, embedding_status |
| document_chunks          | Searchable chunks with optional vectors   | id, document_id, content, section, embedding (JSONB), embedding_model              |
| document_processing_jobs | Durable async queue                       | id, document_id, status, attempts, max_attempts, next_attempt_at, locked_at        |
| financial_metrics        | 26 extracted metrics + evidence           | document_id, fiscal_year, revenue..., metric_evidence (JSONB)                      |
| risk_flags               | Risk findings with source text            | document_id, risk_type, severity, title, source_text, recommendation               |
| analysis_reports         | Generated reports                         | id, user_id, company_id, title, content, disclaimer                                |
| chat_messages            | Research Q&A history                      | session_id, role, content, citations (JSONB)                                       |
| benchmark_comparisons    | Saved benchmark runs                      | id, user_id, payload                                                               |
| agent_logs               | Audit trail of every agent action         | document_id, agent_name, action, status, details, duration_ms                      |

## 7. API reference (20 routes)

| Method + path                              | Purpose                                                    |
|--------------------------------------------|------------------------------------------------------------|
| POST /api/auth/register                    | Create account (bcrypt hash)                               |
| GET/POST /api/auth/[...nextauth]           | Login, logout, session                                     |
| GET/POST /api/documents                    | List / upload documents (triggers pipeline)                |
| GET /api/documents/[id]                    | Document with metrics, risks, chunks, embeddings, activity |
| POST /api/documents/[id]/reprocess         | Re-run the pipeline for a document                         |
| POST /api/documents/[id]/embeddings        | Start/rebuild semantic index                               |
| GET/POST /api/companies                    | List / create companies                                    |
| GET /api/metrics                           | Metric history (cross-period)                              |
| GET /api/risks                             | Risk flags (filterable)                                    |
| GET/POST /api/sessions, /api/sessions/[id] | Research sessions                                          |
| POST /api/chat                             | Research Agent Q&A (citations)                             |
| GET/POST /api/benchmark                    | Benchmark comparisons                                      |
| GET/POST /api/reports, /api/reports/[id]   | Generate and fetch reports                                 |
| GET /api/dashboard                         | Dashboard aggregates                                       |
| GET /api/health                            | Public AI/agent status (no secrets)                        |
| POST /api/worker/documents                 | Cron worker: claim + process job batch (CRON_SECRET)       |
| POST /api/seed                             | Demo data seeding (SEED_SECRET)                            |

## 8. Website guide (every page)

| Page                    | What is on it                                                                                                                | What to demonstrate                                                   |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Home / login / register | Product page and auth forms                                                                                                  | Clean registration flow with validation                               |
| Dashboard               | Counts and recent activity across documents/risks/reports                                                                    | The whole system at a glance                                          |
| Documents (+ upload)    | Upload panel (drag-drop), document table with processing/embedding status                                                    | Upload a PDF and watch the status move to completed                   |
| Document detail         | Content, extracted metrics with evidence, risk flags, chunks, embeddings state, agent activity log (Build embeddings button) | The agent pipeline explained live - each log row is a real stage      |
| Metrics                 | Cross-document metric tables and fiscal-period history                                                                       | 26 metrics with numbers pulled from the documents                     |
| Risks                   | Risk cards grouped by severity with source quotes and recommendations                                                        | Tesla FY2026 demo: four flags with exact filing language              |
| Companies               | Company list with document/metric counts                                                                                     | Multi-company portfolio view                                          |
| Benchmark               | Company selector + comparison table + charts                                                                                 | Apple vs Microsoft vs Tesla on margins and risk counts                |
| Research (chat)         | Question box, streamed answers with [S1] citations, retrieval steps                                                          | The four demo prompts (value, evidence, comparison, follow-up)        |
| Sessions                | Session-scoped research workspaces                                                                                           | Scope a question to one year's documents                              |
| Reports                 | Generated analysis reports + download                                                                                        | Report with deterministic Key Financials table and correct disclaimer |
| /api/health             | JSON status of AI configuration and agents                                                                                   | Live proof of what is configured                                      |

## 9. Setup: local development and cloud deployment

### 9.1 Local development

```
# prerequisites: Node.js 22, npm, a PostgreSQL database
npm install
npm run db:push          # create tables (uses DIRECT_URL/DATABASE_URL)
npm run dev              # http://localhost:3000
```

### 9.2 Environment variables

| Variable           | Purpose                                          | Example / notes                                                                                  |
|--------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------|
| DATABASE_URL       | Pooled Postgres connection (app traffic)         | postgres://... (Neon -pooler- endpoint)                                                          |
| DIRECT_URL         | Unpooled connection (migrations/DDDL)            | postgres://... (Neon direct endpoint)                                                            |
| NEXTAUTH_SECRET    | Session signing secret                           | long random string                                                                               |
| NEXTAUTH_URL       | Public site URL                                  | http://localhost:3000 locally, https://<app> on Render                                           |
| GROQ_API_KEY       | AI for extraction/risk/research/report/benchmark | free key from console.groq.com                                                                   |
| GROQ_MODEL         | Chat model id                                    | openai/ gpt-oss-120b                                                                             |
| EMBEDDING_BASE_URL | Optional: OpenAI-compatible embeddings endpoint  | https:// generativelanguage.googleapis.com/v1beta/openai or http://localhost:11434/v1 for Ollama |
| EMBEDDING_API_KEY  | Optional: key for that endpoint                  | any non-empty value for Ollama                                                                   |
| EMBEDDING_MODEL    | Optional: embedding model id                     | gemini- embedding-001 / nomic-embed-text                                                         |
| CRON_SECRET        | Protects the worker endpoint                     | Bearer token used by the cron job                                                                |
| SEED_SECRET        | Protects the demo-seed endpoint                  | Bearer token                                                                                     |

### 9.3 Deployment (Render + Neon)

- 1. Create a Neon project; copy the pooled URL to DATABASE\_URL and the direct URL to DIRECT\_URL.
- 2. Create a Render Web Service from the GitHub repo: build 'npm ci && npm run build', start 'npm start', Node version pinned by .nvmrc (22).
- 3. Add the environment variables above in the Render dashboard.
- 4. Optional: add a Render Cron Job every 15 minutes: curl -H "Authorization: Bearer \$CRON\_SECRET" https://<app>/api/worker/documents
- 5. Optional: UptimeRobot pings the site every 10 minutes to reduce free-tier cold starts.

*Known platform facts worth stating out loud: free instances cold-start in about a minute; embeddings work only when a provider endpoint is reachable from the cloud (a laptop's Ollama is reachable only from that laptop); uploads are validated by extension and size, and PDF text extraction rejects scanned/image-only PDFs with a clear, actionable error.*

## 10. Security, reliability and graceful degradation

### 10.1 Security

- Passwords are hashed with bcryptjs (salted, slow by design) - plaintext passwords never touch the database.
- Authentication via NextAuth with HTTP-only session cookies; every data query is scoped to session.user.id (users cannot see each other's documents, chats or reports).
- All SQL goes through Drizzle ORM (parameterised queries) - no string-concatenated SQL, no injection surface.
- Server-side validation helpers (src/lib/validation.ts) check UUIDs, file extensions and sizes before anything is stored.
- Secrets (GROQ\_API\_KEY, CRON\_SECRET, SEED\_SECRET...) live only in server environment variables - the client bundle contains none of them.
- Worker and seed endpoints require Bearer-token secrets, so nobody can drain jobs or seed data anonymously.
- AI error messages are sanitised before display (describeNarrativeFallbackReason) so raw stack traces and stale key advice never leak to users.

### 10.2 Graceful degradation (what fails safely)

| Failure                         | What the user sees                                                 | What still works                                                     |
|---------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------|
| Embedding provider missing/down | Embedding status 'unavailable'/'skipped' with explanation          | Keyword search research, all other agents                            |
| Groq AI down or key invalid     | Reports get the deterministic fallback + clean disclaimer          | Metrics (regex), risks (regex/thresholds), keyword research evidence |
| One pipeline stage throws       | Document status 'partial' with the failing stage named in activity | Later stages still run                                               |
| Corrupt/scanned PDF upload      | 422 with the exact parse reason and what to do                     | Everything else                                                      |
| Worker never runs               | Jobs stay queued and are retried when the cron fires               | Documents still saved; processing resumes                            |

## 11. Testing

The project ships with node:test suites runnable via 'npm test' (node --import tsx --test):

| Suite                                   | What it proves                                                              |
|-----------------------------------------|-----------------------------------------------------------------------------|
| src/lib/agents/ extractionUtils.test.ts | Metric label grammar, number/percent parsing, evidence capture              |
| src/lib/agents/ analysisUtils.test.ts   | Red-flag patterns, anomaly thresholds, trend detection, similarity maths    |
| src/lib/agents/ aiStatus.test.ts        | AI fallback reasons name the right key; retired-provider advice never leaks |
| src/lib/agents/ llmClient.test.ts       | Request shaping and error mapping for the AI/Embeddings client              |
| src/lib/ validation.test.ts             | UUID, file type and size validation                                         |
| src/lib/ dbConfig.test.ts               | Pooled vs direct connection selection                                       |
| src/lib/ embeddingUiState.test.ts       | Embedding status surface (configured/unavailable/failed states)             |
| src/lib/ seedPlan.test.ts               | Deterministic demo-seed planning                                            |

## 12. Troubleshooting

| Symptom                                   | Cause                                                            | Fix                                                                        |
|-------------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------|
| 'Could not extract text from this PDF...' | File is corrupt, an error page saved as .pdf, or a scanned image | Re-download/regenerate the file; the error now names the parse reason      |
| Embedding Agent 'skipped/unavailable'     | No EMBEDDING_* configured (Groq has no embeddings API)           | Set the three EMBEDDING_* variables; click Build embeddings                |
| Embedding Agent 'failed' on Render        | EMBEDDING_BASE_URL points at localhost (a laptop-only address)   | Use a cloud endpoint on Render; keep Ollama for local dev only             |
| Reports mention a missing/odd key         | Stale AI error text from an older build                          | Current build maps fallbacks to clean Groq guidance; regenerate the report |
| 'fetch first' on git push                 | Remote and local histories diverged                              | git pull --rebase, then push; never force-push shared branches             |
| Cold start ~60s on Render                 | Free tier sleeps the instance                                    | Expected; UptimeRobot pings reduce it                                      |

## 13. Limitations and future roadmap

- Scanned/image PDFs are rejected by design (no OCR pipeline yet) - roadmap: add OCR (e.g. Tesseract) for scanned filings.
- Vectors use JSONB + in-app cosine - perfect for small/medium collections; roadmap: pgvector with an HNSW index for large corpora.
- Semantic features need an external embedding provider; the system stays fully usable without one (keyword mode).
- Figures come from the documents as written - there is no live market-data feed; roadmap: optional price/quote APIs for valuation ratios.
- AI narratives are grounded and validated but never investment advice; the disclaimer in every report says so.
- Roadmap extras: Docker image, CI/CD with test gates, multi-user organisations/roles, streaming chat tokens, multi-language filings, mobile layout.

## 14. Glossary for beginners

| Term                 | Plain-English meaning                                                                                                       |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Agent                | One module with a single job (read, extract, flag, embed, research, report, benchmark) that the orchestrator calls in order |
| Chunk                | A fixed-size piece of a document (here 1500 characters) stored separately so it can be searched and quoted                  |
| Embedding            | A list of numbers (vector) representing the meaning of a text, so similar texts have similar vectors                        |
| Cosine similarity    | A maths formula measuring how close two vectors are (1.0 = identical meaning direction)                                     |
| RAG                  | Retrieval-Augmented Generation: search the user's documents first, then answer using only what was found                    |
| Grounding            | Keeping an AI answer tied to real source text; here numbers must appear in the retrieved excerpts                           |
| Deterministic        | Code that always produces the same output for the same input (regex, thresholds) - unlike an LLM                            |
| ORM                  | Library that writes SQL for you safely (here Drizzle)                                                                       |
| JSONB                | A Postgres column type storing flexible JSON (used for vectors, evidence, citations)                                        |
| Job queue            | A table of pending work processed in the background so uploads never wait                                                   |
| Cold start           | Delay while a sleeping free-tier server boots up again                                                                      |
| Graceful degradation | When one feature fails, the rest keep working in a reduced but honest mode                                                  |

*End of documentation. The presentation deck (PROJECT\_PRESENTATION.pdf) turns this material into an evaluator-ready walkthrough with demo scripts and code talking points.*

# FinResearch AI

## Multi-Agent Financial Research System

**Evaluation Presentation: architecture, agent logic, code walkthrough, live demo**

**Prepared for project evaluation - beginner friendly, from scratch to full detail**

**October 2026**

**Demo data is illustrative sample data prepared for product demonstration only.**

- > 1. The problem and the solution in one sentence
- > 2. Complete project overview (the end-to-end journey)
- > 3. Technology map: which technology for which feature - and why NOT the alternative
- > 4. The seven agents: logic, code to show, and where the code lives
- > 5. Data model and API surface (what holds it together)
- > 6. Live demo script: exactly what to click on the live website
- > 7. Code walkthrough: which files to open and what to say
- > 8. Security, reliability, cost and testing
- > 9. Limitations (honest) and future scope
- > 10. Evaluator Q&A cheat-sheet

- **Analysing one company properly takes hours: read 100+ pages, copy numbers, scan fine print for warning signs, compare peers, write it up.**

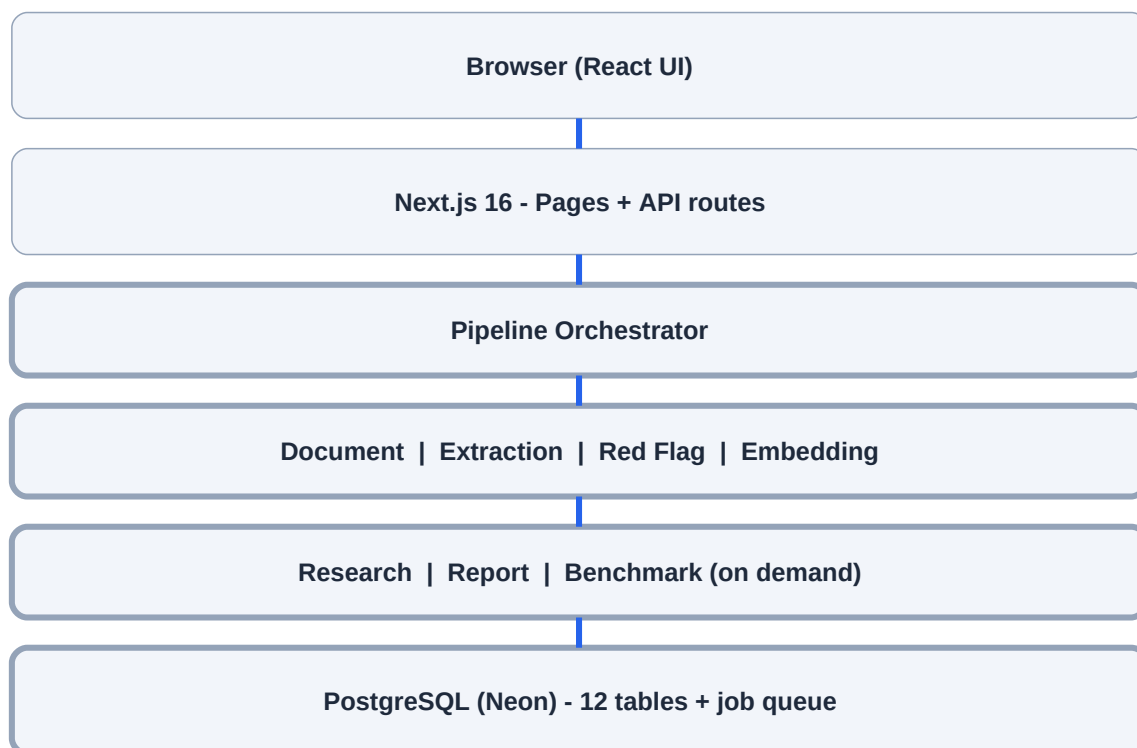
- > Most AI chat tools are fast but UNTRUSTWORTHY for finance: they invent numbers, show no sources, and forget your documents between chats.

- > Analysts, students and investors need both: the speed of AI AND the evidence trail of a spreadsheet.

- > Also: the tooling is fragmented - extraction in one app, comparison in another, report writing in a third.

- **Our one-sentence goal: upload a filing -> get trustworthy, cited financial analysis in minutes.**

- > Upload financial documents (PDF/DOCX/TXT) per company and fiscal year.
- > AUTOMATIC: text indexing (Document Agent) -> 26 metrics with evidence (Extraction Agent) -> risk flags with quotes (Red Flag Agent) -> optional semantic index (Embedding Agent).
- > ON DEMAND: ask research questions with [S1]-style citations (Research Agent), compare companies (Benchmark Agent), generate an analysis report (Report Agent).
- > Every number and every claim carries its source text. If AI is unavailable, deterministic fallbacks keep every feature alive.
- > Runs entirely on free infrastructure: Render + Neon + Groq.



> Upload is fast because processing is a BACKGROUND JOB (durable queue in Postgres) - no request timeouts.

## External services

- Groq AI (LLM): extract, flag, research, report, benchmark narratives
- Embeddings provider (optional): Gemini / Jina / Ollama via 3 env vars
- Render Cron (15 min): drains the job queue
- Secrets stay server-side (env vars only)

| Technology                       | Feature it powers            | Why we chose it                             | Why NOT the alternative                                            |
|----------------------------------|------------------------------|---------------------------------------------|--------------------------------------------------------------------|
| Next.js 16 App Router            | Whole web app (pages + API)  | One codebase, one deploy, SSR, file routing | Vite React: needs a separate backend (2x deploy, CORS/auth pain)   |
| React 19 + TypeScript            | All interactive UI           | Components + type safety at this complexity | Vanilla JS: unmanageable; plain JS: errors reach production        |
| Tailwind CSS 4                   | All styling                  | Fast, consistent, tiny CSS payload          | Bootstrap/MUI: heavy bundles, hard to customise                    |
| Recharts                         | Dashboard + benchmark charts | Declarative React charts from state         | Chart.js: imperative canvas, awkward with React                    |
| Framer Motion                    | Animations                   | Declarative motion for UI polish            | Raw CSS transitions: clumsy for sequences                          |
| react-dropzone + react-hot-toast | Upload UX + notifications    | Accessible DnD; tiny toast library          | Hand-rolled DnD has browser quirks; full alert suites are overkill |

| Technology          | Feature it powers        | Why we chose it                                               | Why NOT the alternative                                           |
|---------------------|--------------------------|---------------------------------------------------------------|-------------------------------------------------------------------|
| Next.js API Routes  | 20 REST endpoints        | Same deploy unit as UI; no CORS; server-side secrets          | Express/Django: second service to deploy, auth must be shared     |
| PostgreSQL (Neon)   | 12-table data model      | Relational analytics for finance; ACID; free serverless tier  | MongoDB: weak joins/aggregation for company-period analysis       |
| Drizzle ORM         | All queries + migrations | TypeScript-first, SQL-like, light, safe (parameterised)       | Prisma: heavier runtime/codegen; raw SQL: injection risk          |
| JSONB vectors       | Semantic search storage  | Vectors in plain Postgres, cosine computed in-app; zero setup | pgvector: better at big scale, needs extension + tuning (roadmap) |
| NextAuth + bcryptjs | Accounts and sessions    | Battle-tested auth; salted slow hashes; HTTP-only cookies     | Clerk/Auth0: paid at scale; hand-rolled JWT: security risk        |
| unpdf + pdf-parse   | PDF text extraction      | Two robust parsers (one as fallback), pure JS                 | Puppeteer/OCR: hundreds of MB RAM - impossible on free tiers      |

| Decision                      | Why we do it THIS way                                                                                    | Why NOT the alternative                                                     |
|-------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Hybrid deterministic + AI     | Numbers must be exact: regex/thresholds extract metrics and flags; AI writes language and adds findings  | Pure-AI invents numbers; pure-rules cannot narrate                          |
| Custom agents (not LangChain) | Each agent = one readable file with explicit I/O, own fallback, own tests - explainable in a viva        | LangChain hides flow behind abstractions; heavy dependencies; hard to debug |
| Groq (openai/ gpt-oss-120b)   | Free tier, very fast inference, OpenAI-compatible API                                                    | OpenAI: paid. Gemini: slower + confusing key errors. Local LLM: needs GPU   |
| Provider-agnostic embeddings  | 3 env vars point at ANY OpenAI-compatible endpoint (Gemini/Jina/Ollama)                                  | Hard-coded vendor = lock-in + single point of failure                       |
| Grounded answers              | Research validates that numbers in answers appear in source excerpts; otherwise return verbatim evidence | Blindly trusting an LLM destroys credibility                                |
| Deterministic report tables   | Key Financials rendered from stored metrics by code                                                      | LLM re-typing numbers = transcription errors                                |

| Technology                | Purpose                                                  | Why / why not the alternative                                                                          |
|---------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Render free web service   | Hosts the Next.js app; auto-deploys from GitHub          | Free + simple for students. Vercel hobby had function/body quirks for this workload; a VPS costs money |
| Neon serverless Postgres  | Database (pooled URL for app, direct URL for DDL)        | Free, autosuspend, pooling built in. Local/RDS Postgres: not free / not reachable                      |
| Render Cron (15 min)      | Calls the protected worker endpoint to drain job retries | Queue services (SQS, Redis/BullMQ): extra accounts, cost and failure points                            |
| Node 22 via .nvmrc        | Pinned runtime                                           | Reproducible builds. Floating versions cause 'works on my machine' bugs                                |
| GitHub + PRs              | Version control and deploy trigger                       | Reviewable history; PR workflow protects main. Zip-over-email is not version control                   |
| Total infrastructure cost | Rs 0 / \$0 per month on free tiers                       | The right kind of project for a student budget                                                         |

| Agent               | One-line job                              | Runs when                      | Code file                                               |
|---------------------|-------------------------------------------|--------------------------------|---------------------------------------------------------|
| 1. Document Agent   | Parse, section, chunk and index the text  | On upload (pipeline)           | src/lib/agents/ documentAgent.ts                        |
| 2. Extraction Agent | Extract 26 metrics + evidence lines       | On upload (pipeline)           | src/lib/agents/ extractionAgent.ts + extractionUtils.ts |
| 3. Red Flag Agent   | Find risks: regex, thresholds, trends, AI | On upload (pipeline)           | src/lib/agents/ riskAgent.ts + analysisUtils.ts         |
| 4. Embedding Agent  | Optional semantic vectors for search      | On upload (pipeline, optional) | src/lib/agents/ embeddingAgent.ts                       |
| 5. Research Agent   | Cited answers from YOUR documents         | When the user asks             | src/lib/agents/ researchAgent.ts                        |
| 6. Report Agent     | Structured analysis report + disclaimer   | When the user asks             | src/lib/agents/ reportAgent.ts                          |
| 7. Benchmark Agent  | Company-vs-company comparisons            | When the user asks             | src/lib/agents/ benchmarkAgent.ts                       |
| Orchestrator        | Runs pipeline stages + job queue          | Always                         | src/lib/agents/ orchestrator.ts                         |

**- Presenter line: 'Each agent is one file, one job, one fallback - that is why the system degrades gracefully instead of crashing.'**

- > Logic: text -> section detection (7 heading patterns) -> sliding-window chunks (1500 chars, 200 overlap) -> document\_chunks rows with section + position.
- > Why overlap: a sentence on a chunk boundary must survive whole in at least one chunk - quotes stay complete.
- > Hybrid search built in: keyword relevance always works; cosine similarity over embeddings when available.

**FOLDER/FILE:** `src/lib/agents/documentAgent.ts`

Show live: Document detail page -> chunks panel

```
export function chunkText(text: string, chunkSize = 1500, overlap = 200): string[] {  
  const chunks: string[] = [];  
  let start = 0;  
  while (start < text.length) {  
    chunks.push(text.slice(start, Math.min(start + chunkSize, text.length)));  
    start += chunkSize - overlap;  
  }  
  return chunks;  
}
```

*What to say: 'This is a sliding window - step size is chunkSize minus overlap, so context never gets surgically cut.'*

- > Hybrid logic (this is the key talking point): (1) deterministic LABELS regex parses 'Total revenue: \$383,285 million' style lines and keeps the matched line as evidence; (2) an LLM pass reads the head+tail of the document (15k chars each) for context; (3) mergeWithLocalMetrics lets the exact regex values win.
- > Normalisation: percents to decimals, money parsed with units, fiscal year detected from labelled lines.
- > If AI is down: step (1) alone still produces the metrics table - the page never goes empty.

**FOLDER/FILE:** `src/lib/agents/extractionAgent.ts + extractionUtils.ts`

Show live: Document detail -> Metrics (26 rows with 'evidence' source line)

```
// extractionUtils.ts -- labelled grammar (excerpt)
revenue: /\s*(?:total net sales|total sales|total net revenues?|
    total revenues?|net sales|net revenues?|revenues?)\s*[:\u2014-]/i

// extractionAgent.ts -- hybrid merge (excerpt)
extracted = mergeWithLocalMetrics(verified, extractMetricsLocally(content));
```

*What to say: 'Regex is used where precision matters; AI is used where language matters. The evidence column proves each number to the audience.'*

- > Layer 1 - regex red flags with severity: auditor qualification (critical), going concern (critical), accounting control (high), liquidity/covenant/default (high). Each stores the matched SOURCE TEXT.
- > Layer 2 - metric anomalies: negative margins, collapsing growth, negative free cash flow (threshold maths on extracted metrics).
- > Layer 3 - cross-period trends: compare with the previous fiscal year of the SAME company (declining margins, rising debt...) and cite BOTH documents.
- > Layer 4 - AI review adds material risks the rules missed - instructed not to duplicate deterministic findings.

**FOLDER/FILE:** `src/lib/agents/riskAgent.ts + analysisUtils.ts`

Show live: Risks page: Tesla FY2026 -> four flags with exact filing quotes

```
// analysisUtils.ts -- deterministic detector (excerpt)
{
  pattern: /\b(?:substantial doubt about.{0,100}going[- ]concern|going[- ]concern)\b/i,
  riskType: "Going Concern", severity: "critical",
  title: "Going-concern disclosure present",
}
```

*What to say: 'Four regex families cover the worst disclosures in audit language. Regex first = zero hallucination risk for the most dangerous claims.'*

- > Batches 32 chunks at a time to ANY OpenAI-compatible /embeddings endpoint (3 env vars: EMBEDDING\_BASE\_URL / EMBEDDING\_API\_KEY / EMBEDDING\_MODEL).
- > Vectors stored in JSONB with the model name; cosine similarity computed in app memory (no pgvector needed).
- > Model versioning: change the model -> chunks re-embed automatically on the next run.
- > No provider -> 'unavailable' with a clear log; research falls back to keyword search. Nothing else breaks.
- > Real-world lesson from this project: Ollama works on your laptop (localhost), a cloud deployment needs a cloud endpoint (Gemini/Jina) - same code, different env vars.

**FOLDER/FILE:** `src/lib/agents/embeddingAgent.ts + llmClient.ts`

Show live: Document detail -> Build embeddings -> status becomes 'completed'

```
// llmClient.ts -- provider-agnostic (excerpt)
await fetch(`${baseUrl}/embeddings`, {
  method: "POST",
  headers: { "content-type": "application/json", authorization: `Bearer ${apiKey}` },
  body: JSON.stringify({ model, input: inputs }),
});
```

*What to say: 'One request format, any provider - that is dependency inversion at the HTTP level.'*

- > Flow: decompose compound question into retrieval steps -> search per step (semantic if embeddings exist, keyword otherwise) -> relevance filter -> buildCitations [S1], [S2]... (document + section + chunk + excerpt).
- > The LLM synthesises ONLY from retrieved passages; validateGroundedResearchAnswer then checks that numbers in the answer appear in the source excerpts.
- > Any AI failure -> return the verbatim passages labelled as EVIDENCE (honest fallback), never a guess.
- > The 'reasoning' shown in the UI lists the retrieval steps that ran - transparency, not hidden chain-of-thought.

**FOLDER/FILE:** `src/lib/agents/researchAgent.ts + analysisUtils.ts`

Show live: Research page: the demo prompts (value / evidence / comparison / follow-up)

```
// researchAgent.ts -- citation ids (excerpt)
citationId: `${index + 1}`

// what the user sees in the answer:
//   ...operating margin was 44.1% [S1]...
//   [S1] Apple FY2023 -- Results of Operations, chunk 2: "..."
```

*What to say: 'Every claim is checkable - the citation opens the exact chunk it came from.'*

- > Sections: Key Financials, Business Overview, Risk Assessment, Financial Analysis, Outlook, Disclaimer.
- > Key Financials table is rendered BY CODE from stored metrics (renderKeyFinancialsSection) - the LLM never re-types numbers.
- > Narrative sections come from the LLM; on failure createLocalReport produces a complete deterministic report.
- > describeNarrativeFallbackReason converts raw AI errors into clean user guidance (this project's bug-fix story: stale GEMINI\_API\_KEY advice used to leak into reports).
- > Reports are downloadable from the Reports page (text/markdown export).

**FOLDER/FILE:** `src/lib/agents/reportAgent.ts` + `aiStatus.ts`

Show live: Reports page -> generate -> show table + disclaimer -> Download

```
// reportAgent.ts -- table stays deterministic (excerpt)
function renderKeyFinancialsSection(profiles) { /* builds the table from stored metrics */ }
// aiStatus.ts -- clean disclaimer mapping (excerpt)
export function describeNarrativeFallbackReason(error?: unknown) { /* maps to Groq guidance */ }
```

*What to say: 'Numbers by code, prose by AI - division of labour that eliminates the classic AI typo risk.'*

- > Builds a metric profile per company (latest or chosen fiscal period), normalises units, then compares: revenue/growth, profitability (margins, ROE), leverage/liquidity (D/E, current ratio), risk counts.
- > Deterministic core: ranking (e.g. by ROE) and N/A handling work without AI.
- > AI narrative is instructed to name the fiscal period beside every figure, refuse silent year-mismatch comparisons, and never give buy/sell advice.

**FOLDER/FILE:** `src/lib/agents/benchmarkAgent.ts`

Show live: Benchmark page: Apple + Microsoft + Tesla side by side

```
// benchmarkAgent.ts -- deterministic ranking (excerpt)
const ranked = [...context].sort((a, b) => Number(b.metrics?.roe || 0) - Number(a.metrics?.roe || 0));
```

*What to say: 'Even with the AI switched off, this page still produces a correct ranking.'*

- > Failure isolation: extraction errors do NOT block risk scanning; embeddings NEVER block anything; the document lands 'completed' or 'partial' with the failing stage named.
- > Durable queue in Postgres: upload inserts a document\_processing\_jobs row; the worker claims it ATOMICALLY.
- > Crash safety: 15-minute stale-lock expiry lets another worker retry; retries use next\_attempt\_at backoff up to max attempts; batch size 5 per run.
- > The worker endpoint is protected by CRON\_SECRET and runs on a 15-minute Render cron (plus an immediate kick after upload for demo speed).

```
// orchestrator.ts -- atomic claim (excerpt)
await db.update(documentProcessingJobs)
  .set({ status: 'processing', attempts: sql`attempts + 1`, lockedAt: now })
  .where(and(
    eq(documentProcessingJobs.id, jobId),
    or(
      and(eq(status, 'queued'), lte(nextAttemptAt, now), lt(attempts, maxAttempts)),
      and(eq(status, 'processing'), lt(lockedAt, staleBefore)) // crashed run recovery
    ),
  ));
```

| Group      | Tables                                                                                | What they hold                                                                               |
|------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Identity   | users, research_sessions                                                              | Accounts (bcrypt hashes) and scoped research workspaces                                      |
| Content    | companies, documents, document_chunks                                                 | Company master data, uploaded files, searchable chunks (+ JSONB vectors)                     |
| Analysis   | financial_metrics, risk_flags, analysis_reports, chat_messages, benchmark_comparisons | 26 metrics + evidence, risks + source quotes, generated reports, cited Q&A, saved benchmarks |
| Operations | document_processing_jobs, agent_logs                                                  | The durable job queue and the per-agent audit trail shown in the UI                          |

> Design points evaluators like: evidence and citations are stored AS DATA (JSONB), not just displayed; every analysis row is tied to a document and a user; agent\_logs makes the whole pipeline auditable after the fact.

| Step | Where (live website)                                                                     | What happens / what to say                                                                                                                                                    |
|------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Register / Login                                                                         | Clean auth flow. 'Passwords are bcrypt-hashed; sessions are HTTP-only cookies.'                                                                                               |
| 2    | Documents -> Upload Apple_Annual_Report_FY2023.pdf (company Apple, ticker AAPL, FY 2023) | Status moves through processing -> completed. 'The upload is fast because agents run as a background job.'                                                                    |
| 3    | Open the document -> agent activity trail                                                | Four stages logged with durations. 'This is the orchestrator's audit trail - every stage is observable.'                                                                      |
| 4    | Document -> Metrics panel                                                                | 26 metrics with evidence lines. 'Numbers came from deterministic parsing; here is the source line for each.'                                                                  |
| 5    | Upload Tesla_Annual_Report_FY2026.pdf -> Risks page                                      | Four flags (auditor qualification, going concern, accounting control, liquidity) with exact quotes. 'Regex detectors - zero hallucination risk on the most dangerous claims.' |

| Step | Where (live website)                          | What happens / what to say                                                                                                           |
|------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 6    | Upload Tesla FY2024 then FY2025 -> Risks page | Cross-period trend findings appear citing BOTH documents. 'Trends are computed by code across periods.'                              |
| 7    | Document detail -> Build embeddings           | Semantic index builds (needs a configured provider). 'Same button works with Ollama locally and Gemini in the cloud.'                |
| 8    | Research page                                 | Ask: 'What evidence supports the going concern risk?' -> answer with [S1] quotes. 'Every sentence is checkable.'                     |
| 9    | Benchmark: Apple + Microsoft + Tesla          | Side-by-side table + charts; Tesla shows the risk spike. 'Periods are named; mismatched years are refused.'                          |
| 10   | Reports -> generate for Tesla                 | Key Financials table + risk narrative + disclaimer. 'Table is rendered by code from stored metrics - the AI never re-types numbers.' |
| 11   | /api/health                                   | JSON shows AI configured, models, embedding status. 'Live configuration transparency.'                                               |

## Code walkthrough (1/2) - open these files in this order

What to show and what to say about each

| File to open                       | Function to point at           | What to say (30 seconds each)                                 |
|------------------------------------|--------------------------------|---------------------------------------------------------------|
| src/lib/agents/ orchestrator.ts    | runDocumentPipeline            | Stage order + failure isolation + 'partial' status design     |
| src/lib/agents/ documentAgent.ts   | chunkText + extractSections    | Sliding window with overlap; regex section detection          |
| src/lib/agents/ extractionUtils.ts | LABELS + extractMetricsLocally | The labelled grammar; evidence capture; numbers normalisation |
| src/lib/agents/ analysisUtils.ts   | detectTextualRedFlags          | The four deterministic patterns and severities                |
| src/lib/agents/ extractionAgent.ts | extractFinancialMetrics        | Hybrid merge: regex values win over AI re-typing              |

| File to open                      | Function to point at                              | What to say (30 seconds each)                                  |
|-----------------------------------|---------------------------------------------------|----------------------------------------------------------------|
| src/lib/agents/ researchAgent.ts  | buildCitations +<br>buildLocalResearchAnswer      | [S1] citations; verbatim-evidence fallback instead of guessing |
| src/lib/agents/ analysisUtils.ts  | validateGroundedResearchAnswer                    | Numbers in answers must appear in source excerpts              |
| src/lib/agents/ embeddingAgent.ts | indexDocumentEmbeddings                           | Batching, model versioning, optional-by-design                 |
| src/lib/agents/ reportAgent.ts    | renderKeyFinancialsSection +<br>createLocalReport | Deterministic tables; deterministic fallback report            |
| src/lib/agents/aiStatus.ts        | describeNarrativeFallbackReason                   | User-safe error mapping (the GEMINI->Groq bug-fix story)       |
| src/app/api/documents/route.ts    | PDF extraction block                              | Two-parser fallback + actionable error messages                |

- > Passwords: bcryptjs (salted, slow hashes). Auth: NextAuth with HTTP-only cookies. No third-party lock-in.
- > Data isolation: every query is scoped to `session.user.id` - users cannot read each other's documents, chats or reports.
- > Injection-safe: all SQL through Drizzle ORM (parameterised); server-side validation for UUIDs, file types and sizes.
- > Secrets (`GROQ_API_KEY`, `CRON_SECRET`, `SEED_SECRET`, `EMBEDDING_API_KEY`) exist only in server environment variables - nothing sensitive in the browser bundle.
- > Protected operations: worker and seed endpoints require Bearer tokens; cron calls are authenticated.
- > Safe output: AI errors are sanitised before display; reports carry an explicit 'not investment advice' disclaimer.
- > Demo integrity: all sample documents are clearly labelled 'SAMPLE DOCUMENT - illustrative figures'.

| If this fails...                   | The user sees...                           | Everything that still works                    |
|------------------------------------|--------------------------------------------|------------------------------------------------|
| Embeddings provider missing/down   | 'unavailable' + explanation (not an error) | Keyword research, metrics, risks, reports      |
| Groq AI down / key invalid         | Deterministic report + clean disclaimer    | Regex metrics + flags, keyword evidence search |
| One pipeline stage throws          | Status 'partial' naming the stage          | Later stages still run; activity log explains  |
| Corrupt or scanned PDF             | 422 naming the exact parse reason          | All other documents and features               |
| Cloud Ollama misconfig (localhost) | Embedding 'failed' with connection error   | Everything except semantic search              |

- > Upload speed: processing is asynchronous (job queue) - the browser never waits for AI calls.
- > AI budget control: prompts take head+tail of documents (15k chars each) instead of whole files; embedding batches of 32; 8k chars max per chunk.
- > Storage: JSONB vectors + in-app cosine suit small/medium corpora; pgvector is the scale-up path (roadmap).
- > Free-tier aware: Render free cold starts (~1 min) are mitigated with an uptime monitor; Neon autosuspends when idle.
- > Total monthly infrastructure cost for the demo: Rs 0 / \$0 (Render free + Neon free + Groq free).

| Suite                                       | What it proves to an evaluator                                                  |
|---------------------------------------------|---------------------------------------------------------------------------------|
| extractionUtils.test.ts                     | The metric grammar parses real filing styles and stores evidence                |
| analysisUtils.test.ts                       | Red-flag patterns, thresholds, trend detection and similarity maths are correct |
| aiStatus.test.ts                            | Fallbacks name the right provider key; retired advice (GEMINI) never leaks      |
| llmClient.test.ts                           | AI/Embeddings request shaping and error mapping                                 |
| validation.test.ts / dbConfig.test.ts       | Input validation and pooled-vs-direct connection logic                          |
| embeddingUiState.test.ts / seedPlan.test.ts | Embedding status UI logic; deterministic demo seeding                           |

| Limitation today                     | Why it is acceptable                              | Future scope                                    |
|--------------------------------------|---------------------------------------------------|-------------------------------------------------|
| No OCR for scanned/image PDFs        | Clear, actionable error instead of silent garbage | OCR pipeline (Tesseract) for scanned filings    |
| Vectors in JSONB + in-app cosine     | Correct and simple at small/medium scale          | pgvector + HNSW index for large corpora         |
| Embeddings need an external provider | Keyword mode keeps ALL features usable            | Ship a default provider / local model option    |
| No live market prices                | Analysis is document-based and traceable          | Optional quote/valuation API integration        |
| AI can still err (rarely)            | Grounding validation + citations + disclaimers    | Stronger claim-level verification; streaming UX |
| Single-user workspaces               | Fits the student/analyst use case                 | Organisations, roles, sharing; Docker + CI/CD   |

| Question they ask                          | The answer to give                                                                                                                                                                                           |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why multi-agent instead of one big prompt? | Each agent has one job, one test surface and one fallback. Failures stay isolated, the pipeline is auditable (agent_logs), and precision work stays deterministic.                                           |
| How do you control hallucination?          | Three defences: deterministic extraction for numbers; citations with source excerpts; grounding validation that checks numbers appear in retrieved text. Worst case: verbatim evidence instead of an answer. |
| Why not LangChain / CrewAI?                | This scale does not need them. Explicit TS files are easier to explain, test, debug and deploy; fewer dependencies, smaller bundle.                                                                          |
| What if the AI provider dies?              | Every feature has a deterministic fallback (regex metrics/flags, keyword search, local report/benchmark). The UI explains the mode instead of failing.                                                       |

| Question they ask                     | The answer to give                                                                                                                                                                      |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Is it secure?                         | Bcrypt hashes, HTTP-only sessions, per-user query scoping, ORM parameterisation, server-side secrets, Bearer-protected worker/seed endpoints, sanitised error messages.                 |
| How does it scale?                    | Async jobs + batching + prompt trimming cover the free tier; the documented scale-up path is pgvector, queue hardening and caching. Design decisions are deliberate, not accidental.    |
| What did YOU learn building it?       | Mention: the GEMINI->Groq disclaimer bug (trust!), the 'saved 404 page as PDF' debugging story (errors must self-diagnose), and localhost-vs-cloud Ollama (environment config matters). |
| Is this financial advice?             | No - it is research assistance with evidence. Every report carries a disclaimer; benchmark refuses buy/sell language by design.                                                         |
| What would you add with 2 more weeks? | OCR, pgvector, streaming chat, Docker+CI, multi-user roles, and a chart-heavy valuation module.                                                                                         |

# Thank you

FinResearch AI - trustworthy financial research, one upload away.

Questions welcome. Live demo available at the deployed website.

---

Documentation: [PROJECT\\_DOCUMENTATION.pdf](#) | Demo data: [demo-test-data/](#) (28 sample annual reports)